

ASP.NET Core Web API — конспект за сьогодні

1. Що ми будуємо

За ТЗ потрібно зробити:

```
FlightStorageService (Web API)
    ↓ REST
FlightClientApp (UI)
```



Поки що ми працюємо тільки над:

```
FlightStorageService
```



2. Структура проєкту

Зараз у тебе є:

```
FlightStorageService
|
├── Controllers
|   └── FlightsController.cs
|
├── Models
|   └── Flight.cs
|
├── Services
|   ├── IFlightService.cs
|   └── FlightService.cs
|
├── Repositories
|   ├── IFlightRepository.cs
|   └── FlightRepository.cs
|
├── Program.cs
|
├── appsettings.json
└── appsettings.Development.json
```



3. Model

Файл:

Models/Flight.cs



Модель описує дані.

</> C#



```
public class Flight
{
    public string FlightNumber { get; set; } = string.Empty;

    public DateTime DepartureDateTime { get; set; }

    public string DepartureAirportCity { get; set; } = string.Empty;

    public string ArrivalAirportCity { get; set; } = string.Empty;

    public int DurationMinutes { get; set; }
}
```

Це майже один в один таблиця з БД.

4. Controller

Файл:

Controllers/FlightsController.cs



Контролер приймає HTTP-запити.

Наприклад:

</> http



```
GET /api/flights/test
```

потрапляє сюди:

</> C#



```
[HttpGet("test")]
public IActionResult Test()
{
}
```

Контролер НЕ повинен:

✗ Працювати з SQL

✗ Робити складну бізнес-логіку

Його задача:

```
Прийняти запит
    ↓
Викликати сервіс
    ↓
Повернути результат
```



5. Service

Файли:

```
Services/IFlightService.cs
Services/FlightService.cs
```



Тут живе бізнес-логіка.

Наприклад із ТЗ:

```
</> C#

if(date > DateTime.Today.AddDays(7))
{
    throw ...
}
```



або

```
</> C#

if(string.IsNullOrWhiteSpace(city))
{
    throw ...
}
```



Сервіс відповідає на питання:

```
Що робити?
```



6. Repository

Файли:

```
Repositories/IFlightRepository.cs  
Repositories/FlightRepository.cs
```



Репозиторій відповідає за дані.

Зараз:

```
</> C#
```



```
return new Flight(...);
```

Пізніше:

```
</> C#
```



```
SqlConnection  
SqlCommand  
Stored Procedure
```

Repository відповідає на питання:

```
ЗВІДКИ взяти дані?
```



7. Архітектура

Було:

```
Controller  
↓  
Flight
```



Стало:

```
Controller  
↓  
Service  
↓  
Repository  
↓  
Flight
```



У майбутньому:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
SQL Server
```



8. Dependency Injection (DI)

Найважливіша тема дня.

ASP.NET сам створює потрібні об'єкти.

Реєстрація:

</> C#



```
builder.Services.AddScoped<IFlightService, FlightService>();

builder.Services.AddScoped<IFlightRepository, FlightRepository>();
```

Це означає:

```
IFlightService
  ↓
FlightService

IFlightRepository
  ↓
FlightRepository
```



Після цього ASP.NET може автоматично створювати об'єкти.

9. Constructor Injection

У контролері:

</> C#



```
private readonly IFlightService _flightService;

public FlightsController(IFlightService flightService)
{
    _flightService = flightService;
}
```

ASP.NET бачить:

Потрібен IFlightService



Дивиться у DI:

IFlightService → FlightService



Створює його автоматично.

Те саме у сервісі:

</> C#



```
private readonly IFlightRepository _flightRepository;

public FlightService(IFlightRepository flightRepository)
{
    _flightRepository = flightRepository;
}
```

10. Потік виконання

Коли відкриваєш:

/api/flights/test



відбувається:

Browser



FlightsController



↓
FlightService
↓
FlightRepository
↓
Flight
↓
JSON
↓
Browser

11. JSON Serialization

Коли повертаєш:

</> C#



```
return Ok(flight);
```

ASP.NET автоматично робить:

Flight Object



↓
JSON

Результат:

</> JSON



```
{  
  "flightNumber": "PS123",  
  "departureDateTime": "...",  
  "departureAirportCity": "Kyiv",  
  "arrivalAirportCity": "London",  
  "durationMinutes": 180  
}
```

12. Що треба зрозуміти перед SQL

Перш ніж лізти в SQL, потрібно розібрати:

Route Parameters

```
/api/flights/PS123
```



```
</> C#
```



```
[HttpGet("{flightNumber}")]
```

Query Parameters

```
/api/flights?date=2026-06-11
```



```
</> C#
```



```
[HttpGet]
```

Кілька Query Parameters

```
/api/flights/departure?city=Kyiv&date=2026-06-11
```



Що вже знаєш після сьогоднішнього заняття

- ✓ Структура ASP.NET Core проєкту
 - ✓ Model
 - ✓ Controller
 - ✓ Service
 - ✓ Repository
 - ✓ Dependency Injection
 - ✓ Constructor Injection
 - ✓ JSON Serialization
 - ✓ Робота першого API endpoint
-

Наступний логічний блок навчання:

Routes

↓

Query Parameters

↓

Validation

↓

SQL Server

↓

ADO.NET

↓

Stored Procedures



Після маршрутів ти вже почнеш реалізовувати реальні ендпоінти з технічного завдання. 🚀